

ABSTRACT

This document details the development of a Todo Management application using the Django framework. The project is structured into two main directories: `app` and `todo_project`. The `app` directory contains essential Django components such as models, views, forms, and templates, while `todo_project` holds configuration and routing files.

The application's primary functionality revolves around managing Todo items for authenticated users. Users can sign up, log in, and perform CRUD (Create, Read, Update, Delete) operations on their Todo items. The Todo items have attributes such as title, status, priority, and associated user, and are ordered by priority.

The database schema design is critical for this application, involving tables for users and Todo items. The User model is provided by Django's authentication system, while the Todo model is custom-defined with fields for title, status, user, date, and priority. Relationships between tables are established through foreign keys, ensuring data integrity and proper linkage.

This document also includes an Entity-Relationship (ER) diagram that visually represents the database schema. The diagram highlights the tables, their fields, and the relationships between them, providing a clear overview of the logical structure of the database.

INTRODUCTION

The Todo Management application is designed to streamline task management for users by allowing them to create, track, and organize their daily tasks efficiently. Built using the Django framework, this application leverages the robust features of Django to ensure a secure, scalable, and maintainable solution. The primary objective is to provide a user-friendly interface where users can sign up, log in, and manage their tasks with ease. This application is particularly beneficial for individuals looking to enhance their productivity and keep their tasks organized in a systematic manner.

The project architecture is meticulously structured into two main directories: `app` and `todo_project`. The `app` directory encapsulates the core components of the Django application, including models, views, forms, and templates. These components work cohesively to handle the business logic, data representation, and user interactions within the application. The `todo_project` directory, on the other hand, houses configuration files that manage the overall project settings, including URL routing and application-wide configurations. This separation of concerns ensures that the application remains modular and easy to manage.

A key aspect of the application is its database schema design, which involves creating tables and defining relationships between them. The application uses Django's built-in User model to handle authentication, while a custom Todo model is implemented to store task details. The Todo model includes fields for the title, status, priority, and associated user, along with timestamps to track the creation date. These models are interconnected through foreign key relationships, ensuring data integrity and facilitating efficient data retrieval. An Entity-Relationship (ER) diagram visually represents this schema, providing a clear overview of the database structure and relationships, which is essential for understanding and extending the application's functionality.

SOFTWARE REQUIREMENTS

1. **Operating System:** The application can be developed and run on any operating system that supports Python, including Windows, macOS, and Linux.
2. **Python:** Python 3.6 or higher is required for running Django.
3. **Django:** Django 3.2 or higher is recommended for the application.
4. **Database:** SQLite (default for Django) or any other database supported by Django (e.g., PostgreSQL, MySQL).
5. **Web Browser:** A modern web browser (e.g., Chrome, Firefox, Safari) for accessing the application.
6. **Text Editor/IDE:** An IDE or text editor such as VS Code, PyCharm, or Sublime Text for writing and managing code.

HARDWARE REQUIREMENTS

1. **Processor:** A multi-core processor (e.g., Intel i5/i7, AMD Ryzen) is recommended for efficient development and running of the application.
2. **RAM:** At least 8GB of RAM is recommended for smooth operation, especially when running multiple services and development tools simultaneously.
3. **Storage:** A minimum of 256GB SSD is recommended for faster read/write operations. More storage may be required depending on the size of the database and project files.
4. **Display:** A monitor with a resolution of at least 1920x1080 is recommended for better visibility and ease of development.
5. **Network:** A stable internet connection is necessary for downloading dependencies, accessing online resources, and deploying the application.

TECHNICAL STACK

Backend (Django)

- **Django:** Use Django as the primary framework. It offers a robust structure for creating models, views, and templates.
- **Django REST Framework (DRF):** If you want to add API endpoints for a future frontend or mobile app, DRF can make it easier to manage data through APIs.
- **SQLite:** The default Django database is simple and effective for local development, requiring no additional setup.

Frontend (Templates)

- **Django Templates:** Use Django's templating system to dynamically render HTML content based on backend data.
- **HTML and CSS:** For the app's structure and styling. Use these to create a clear and functional interface.
- **JavaScript:** Add interactivity to the app, such as handling task status updates without a full page reload.
- **Tailwind:** Use a CSS framework to make the design more appealing and responsive without heavy customization.

FRONTEND CODE

base.html

```
<!doctype html>
<html lang="en">
<head>
  <!-- Required meta tags -->
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
  <!-- Bootstrap CSS -->
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.5.3/dist/css/bootstrap.min.css"
  integrity="sha384-
TX8t27EcRE3e/ihU7zmQxVncDAy5uIKz4rEkgIXeMed4M0jlfIDPvg6uqKI2xXr2"
crossorigin="anonymous">
  <title>TODO APP</title>
</head>
<body>
  <!-- navbar start -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <a class="navbar-brand" href="/">TODO</a>
    <button class="navbar-toggler" type="button" data-toggle="collapse" data-
target="#navbarNav"
      aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav ml-auto">
        {% if request.user.is_authenticated %}
        <li class="nav-item">
          <a class="nav-link" href="/logout">Logout</a>
        </li>
```

```

    {% else %}
    <li class="nav-item">
      <a class="nav-link" href="/login12">Login</a>
    </li>
    <li class="nav-item">
      <a class="nav-link" href="/signup12/">Signup</a>
    </li>
    {% endif %}
  </ul>
</div>
</nav>
<!-- changing coontent -->
<div>
  {% block content %}
  {% endblock %}
</div>
<!-- end -->
<script src="https://code.jquery.com/jquery-3.5.1.slim.min.js"
  integrity="sha384-
DfXdz2htPH0lsSSs5nCTpuj/zy4C+OGpamoFVy38MVBnE+IbbVYUew+OrCXaRkfj"
  crossorigin="anonymous"></script>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@4.5.3/dist/js/bootstrap.bundle.min.js"
  integrity="sha384-
ho+j7jyWK8fNQe+A12Hb8AhRq26LrZ/JpcUGGOn+Y7RsweNrtN/tE3MoK7ZeZDyx"
  crossorigin="anonymous"></script>
</body>
</html>

```

index.html

```

{% extends 'base.html' %}

{% block content %}

```

```

<div>
  <div class="container pt-4">
    <div class="row p-0 mt-4">
      <div class="col-lg-4 mx-auto shadow p-0">
        <div class="alert alert-info">
          <h2>Add New TODO</h2>
        </div>
        <div class="p-4">
          <form action="/add-todo/" method="POST">
            {% csrf_token %}
            {{form.as_p}}
            <hr>
            <input type="submit" value="ADD" class="btn btn-info">
          </form>
        </div>
      </div>
    </div>
    <div class="col">
      <div class="border">
        {% if todos|length == 0 %}
          <div class="p-4">
            <br>
            <br>
            <div class="alert alert-info text-center">
              <p class="" style="font-size: 30px;">No Todos</p>
            </div>
            <br>
            <br>
          </div>
        {%else%}
          <div class="alert alert-info">
            {% if user_name %}
              <h4>Welcome {{ user_name }}</h4>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
</div>

```

```

{% endif %}
</div>
<div>
<table class="table">
<thead>
<tr>
<th>#</th>
<th>Title</th>
<th>Status</th>
<th>Priority</th>
<th>Action</th>
</tr>
</thead>
<tbody>
{% for todo in todos %}
<tr>
<td>{{forloop.counter}}</td>
<td>{{todo.title}}</td>
<td>
{% if todo.status == 'C'%}

{%else%}

{%endif%}
</td>
<td>{{todo.priority}}</td>
<td>
<a href="/delete-todo/{{todo.id}}" title="Delete" class="">  </a>
{% if todo.status == 'P'%}
<a href="/change-status/{{todo.id}}/C" title="Mark Completed" class="">  </a>
{%else%}

```

```

        <a href="/change-status/{{todo.id}}/P" title="Mark Pending" class="">🔴 </a>
    {%endif%}
</td>
</tr>
{%endfor%}
</tbody>
</table>
</div>
{%endif%}
</div>
</div>
</div>
</div>
</div>
{% endblock %}

```

login.html

```

{% extends 'base.html' %}
{% block content %}
<div class="container pt-5">
    <div class="col-lg-4 mx-auto shadow p-0 mt-10">
        <div class="alert alert-info">
            <h2>Login</h2>
        </div>
        <div class="p-4">
            <form method="post" action="/login12/">
                {% csrf_token %}
                {{ form.as_p }}
                <hr>
                <button type="submit" value="Login" class="btn btn-info">Login</button>
            </form>
        </div>
    </div>
</div>

```

```
</form>
</div>
</div>
</div>
{% endblock %}
```

signup.html

```
{% extends 'base.html' %}
{% block content %}
<div class="containter pt-4">
  <div class="col-lg-5 shadow mx-auto p-0">
    <div class="alert alert-success">Create Account</div>
    <div class="p-4">

      <form method="post" action="/signup12/" >
        {% csrf_token %}
        {{ form.as_p }}
        <hr>
        <input class="btn btn-success" type="submit" value="Create Account">
      </form>
    </div>
  </div>
</div>
{% endblock %}
```

BACKEND CODE :

admin.py

```
from django.contrib import admin
from app.models import TODO
```

```
admin.site.register(TODO)
```

apps.py

```
from django.apps import AppConfig

class AppConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'app'
```

form.py

```
from django.forms import ModelForm
from app.models import TODO
class TODOForm(ModelForm):
    class Meta:
        model = TODO
        fields = ['title', 'status', 'priority'] #date,user,date
```

models.py

```
from django.db import models
from django.contrib.auth.models import User
```

```
class TODO(models.Model):
    status_choices = [
        ('C', 'COMPLETED'),
        ('P', 'PENDING'),
    ]
    priority_choices = [
        ('1', '1'),
        ('2', '2'),
        ('3', '3'),
        ('4', '4'),
        ('5', '5'),
        ('6', '6'),
        ('7', '7'),
        ('8', '8'),
        ('9', '9'),
```

```

('10', '10'),
]
title = models.CharField(max_length=50)
status = models.CharField(max_length=2, choices=status_choices)
user = models.ForeignKey(User, on_delete=models.CASCADE)
date = models.DateTimeField(auto_now_add=True)
priority = models.CharField(max_length=2, choices=priority_choices)

```

urls.py

```

from django.contrib import admin
from django.urls import path
# from django.shortcuts import HttpResponseRedirect
from app.views import *
# def home(request):
#     # print("Home")
#     return HttpResponseRedirect("hello")

urlpatterns = [
    path("", home, name='home'),
    path('login12/', login1, name='login11'),
    path('signup12/', signup1, name='signup11'),
    path('add-todo/', add),
    path('delete-todo/<int:id>', dele),
    path('change-status/<int:id>/<str:status>', cha_sta),
    path('logout/', signout),
]

```

views.py

```

from django.shortcuts import render, redirect
from django.shortcuts import HttpResponseRedirect
from django.contrib.auth.forms import UserCreationForm, AuthenticationForm
from django.contrib.auth import authenticate, login as loginUser, logout

```

```

from app.form import *
from django.contrib.auth.decorators import login_required
@login_required(login_url='login11')
def home(request):
    if request.user.is_authenticated:
        user = request.user
        form = TODOForm()
        todos = TODO.objects.filter(user = user).order_by('priority')
        user_name = user.username if user else None # Get the user's name
        return render(request, 'index.html', context={'form': form, 'todos': todos, 'user_name':
user_name})
def login1(request):
    if request.method == 'GET':
        form1 = AuthenticationForm()
        context = {
            "form" : form1
        }
        return render(request , 'login.html' , context=context )
    else:
        form = AuthenticationForm(data=request.POST)
        print(form.is_valid())
        if form.is_valid():
            username = form.cleaned_data.get('username')
            password = form.cleaned_data.get('password')
            user = authenticate(username = username , password = password)
            if user is not None:
                loginUser(request , user)
                return redirect('home')
        else:
            context = {
                "form" : form
            }

```

```

        return render(request , 'login.html' , context=context )
def signup1(request):
    if request.method == 'GET':
        form = UserCreationForm()
        context = {
            "form" : form
        }
        return render(request , 'signup.html' , context=context)
    else:
        print(request.POST)
        form = UserCreationForm(request.POST)
        context = {
            "form" : form
        }
        if form.is_valid():
            user = form.save()
            print(user)
            if user is not None:
                return redirect('login11')
        else:
            return render(request , 'signup.html' , context=context)
@login_required(login_url='login11')
def add(request):
    if request.user.is_authenticated:
        user = request.user
        print(user)
        form = TODOForm(request.POST)
        if form.is_valid():
            print(form.cleaned_data)
            todo = form.save(commit=False)
            todo.user = user
            todo.save()

```

```

        print(todo)
        return redirect("home")
    else:
        return render(request , 'index.html' , context={'form' : form})
def dele(request , id ):
    print(id)
    TODO.objects.get(pk = id).delete()
    return redirect('home')
def cha_sta(request , id , status):
    todo = TODO.objects.get(pk = id)
    todo.status = status
    todo.save()
    return redirect('home')
def signout(request):
    logout(request)
    return redirect('login11')

```

settings.py

"""

Django settings for todo_project project.

Generated by 'django-admin startproject' using Django 5.0.6.

For more information on this file, see

<https://docs.djangoproject.com/en/5.0/topics/settings/>

For the full list of settings and their values, see

<https://docs.djangoproject.com/en/5.0/ref/settings/>

"""

```

from pathlib import Path

```

```
# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent

# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/5.0/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'django-insecure-
u)4@@9vu8bwm_brwt_34@r&l0%pcqy5%f7%flv@hezme6sy5%9'

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = []

# Application definition

INSTALLED_APPS = [
    'app',
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
```

```

'django.contrib.auth.middleware.AuthenticationMiddleware',
'django.contrib.messages.middleware.MessageMiddleware',
'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'todo_project.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    ],
]

```

```
WSGI_APPLICATION = 'todo_project.wsgi.application'
```

```
# Database
```

```
# https://docs.djangoproject.com/en/5.0/ref/settings/#databases
```

```

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

```

```

    }
}

# Password validation
# https://docs.djangoproject.com/en/5.0/ref/settings/#auth-password-validators

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]

# Internationalization
# https://docs.djangoproject.com/en/5.0/topics/i18n/

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'

USE_I18N = True

USE_TZ = True

```

```
# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/5.0/howto/static-files/
```

```
STATIC_URL = 'static/'
```

```
# Default primary key field type
# https://docs.djangoproject.com/en/5.0/ref/settings/#default-auto-field
```

```
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```

todo_project/urls.py

```
from django.contrib import admin
from django.urls import path, include
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('app.urls'))
]
```

wsgi.py

```
"""
```

WSGI config for todo_project project.

It exposes the WSGI callable as a module-level variable named ``application``.

For more information on this file, see

<https://docs.djangoproject.com/en/5.0/howto/deployment/wsgi/>

```
"""
```

```
import os
```

```
from django.core.wsgi import get_wsgi_application
```

```
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'todo_project.settings')
```

```
application = get_wsgi_application()
```

manage.py

```
#!/usr/bin/env python
```

```
"""Django's command-line utility for administrative tasks."""
```

```
import os
```

```
import sys
```

```
def main():
```

```
    """Run administrative tasks."""
```

```
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'todo_project.settings')
```

```
    try:
```

```
        from django.core.management import execute_from_command_line
```

```
    except ImportError as exc:
```

```
        raise ImportError(
```

```
            "Couldn't import Django. Are you sure it's installed and "
```

```
            "available on your PYTHONPATH environment variable? Did you "
```

```
            "forget to activate a virtual environment?"
```

```
        ) from exc
```

```
    execute_from_command_line(sys.argv)
```

```
if __name__ == '__main__':
```

```
    main()
```

SCREEN SHOTS

Login

Username:

Password:

Create Account

Username: Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password:

- Your password can't be too similar to your other personal information.
- Your password must contain at least 8 characters.
- Your password can't be a commonly used password.
- Your password can't be entirely numeric.

Password confirmation: Enter the same password as before, for verification.

Add New TODO

Title: Status: Priority:

ADD

Welcome abhi@9275

#	Title	Status	Priority	Action
1	qwe	🕒	1	🗑️✅
2	asd	🕒	3	🗑️✅
3	ghfgh	✅	3	🗑️🕒
4	sfgh	🕒	8	🗑️✅

Add New TODO

Title: Status: Priority:

ADD

Welcome Mani@9275

#	Title	Status	Priority	Action
1	WF	✅	1	🗑️🕒
2	sdfs	✅	6	🗑️🕒